

▼ Módulo 4: Conversão de Tipos

Certifique-se de que você está dentro da pasta da aula antes de executar o comando abaixo. O comando criará automaticamente as subpastas `cpp/src`, `cpp/bin` e `cpp/lib` dentro dela. Caso tenha executado anteriormente, não é necessário reexecutar.

```
!mkdir -p cpp/src cpp/bin cpp/lib
```

253.32s - pydevd: Sending message related to process being replaced timed-out after 5 seconds

A estrutura gerada será a seguinte.

NOME_DA_AULA/

```
├─ cpp/
│   ├── src/
│   ├── bin/
│   └─ lib/
```

Pasta	Conteúdo	Extensões
<code>src</code>	Arquivos fonte com a lógica do programa	<code>.cpp</code>
<code>bin</code>	Executáveis gerados pelo compilador	sem extensão
<code>lib</code>	Cabeçalhos e bibliotecas externas	<code>.h</code> <code>.hpp</code>

Nota sobre os comandos do Jupyter Notebook

Você vai notar que os exemplos desta aula utilizam alguns comandos especiais, como `%%writefile`, `!g++` e `!./...`. É importante entender que **esses prefixos não fazem parte do C++**, O `%%writefile` e o `!` são apenas uma "ponte" entre o Jupyter e o terminal do sistema. **O fluxo real é o mesmo que você vê nos vídeos:** escrever o código, compilar com `g++` e executar o binário. — eles são recursos do próprio Jupyter Notebook que nos permitem escrever e executar código C++ diretamente por aqui.

- `%%writefile cpp/src/02_exec_02.cpp` salva o conteúdo da célula como um arquivo `.cpp` no disco.
- `!g++ cpp/src/02_exec_02.cpp -o cpp/bin/02_exec_02` compila o arquivo usando o `g++`, exatamente como você faria no terminal.
- `!./cpp/bin/02_exec_02` executa o binário gerado.

Disponibilizamos os exemplos dessa forma para que você consiga acompanhar e rodar tudo diretamente na aula, sem precisar sair do notebook, de modo online e interativo. Mas recomendamos que, sempre que possível, **pratique também pelo terminal**, como demonstrado nos vídeos — é assim que você vai trabalhar no dia a dia com C++.

▼ 4.1. Conversão Implícita

Conversão implícita é uma conversão de tipo realizada automaticamente pelo compilador, sem nenhuma instrução explícita do desenvolvedor. Ela acontece porque o C++ é uma linguagem fortemente tipada, ou seja, toda operação exige que os operandos falem a mesma língua. Quando isso não ocorre, o compilador resolve sozinho, promovendo o tipo de menor capacidade para o tipo de maior capacidade antes de executar a operação.

Ela existe porque no mundo real raramente trabalhamos com variáveis de um único tipo. Preços são `double`, quantidades são `int`, flags são `bool`. O compilador precisa de uma estratégia consistente para lidar com essas misturas sem que o programa quebre.

4.1.1 A Hierarquia de Promoção

A regra central é simples: o tipo de menor precisão sempre é promovido para o tipo de maior precisão. A ordem, do menor para o maior, é a seguinte.

```
bool → char → short → int → long → float → double → long double
```

Quando dois tipos diferentes se encontram em uma expressão, o compilador identifica qual dos dois está mais à esquerda nessa hierarquia e o converte para o tipo do outro antes de executar o cálculo.

Tabela de Tipos Sem Modificadores

Tipo	Tamanho	Valor Mínimo	Valor Máximo
<code>bool</code>	1 byte	0 (false)	1 (true)
<code>char</code>	1 byte	-128	127
<code>short</code>	2 bytes	-32.768	32.767
<code>int</code>	4 bytes	-2.147.483.648	2.147.483.647
<code>long</code>	4 ou 8 bytes	-2.147.483.648	2.147.483.647

Tipo	Tamanho	Valor Mínimo	Valor Máximo
<code>float</code>	4 bytes	$\sim 1.2 \times 10^{-38}$	$\sim 3.4 \times 10^{38}$
<code>double</code>	8 bytes	$\sim 2.3 \times 10^{-308}$	$\sim 1.7 \times 10^{308}$
<code>long double</code>	10 a 16 bytes	$\sim 3.4 \times 10^{-4932}$	$\sim 1.1 \times 10^{4932}$

Tabela de Tipos Com Modificadores

Os modificadores `signed`, `unsigned`, `short`, `long` e `long long` alteram o intervalo de valores sem mudar o número de bytes. O ponto mais importante é que o modificador `unsigned` elimina os valores negativos e dobra o limite superior.

Tipo Completo	Tamanho	Valor Mínimo	Valor Máximo
<code>bool</code>	1 byte	0 (false)	1 (true)
<code>char</code>	1 byte	-128	127
<code>signed char</code>	1 byte	-128	127
<code>unsigned char</code>	1 byte	0	255
<code>short</code>	2 bytes	-32.768	32.767
<code>unsigned short</code>	2 bytes	0	65.535
<code>int</code>	4 bytes	-2.147.483.648	2.147.483.647
<code>unsigned int</code>	4 bytes	0	4.294.967.295
<code>long</code>	4 ou 8 bytes	-2.147.483.648	2.147.483.647
<code>unsigned long</code>	4 ou 8 bytes	0	4.294.967.295
<code>long long</code>	8 bytes	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
<code>unsigned long long</code>	8 bytes	0	18.446.744.073.709.551.615
<code>float</code>	4 bytes	$\sim 1.2 \times 10^{-38}$	$\sim 3.4 \times 10^{38}$
<code>double</code>	8 bytes	$\sim 2.3 \times 10^{-308}$	$\sim 1.7 \times 10^{308}$
<code>long double</code>	10 a 16 bytes	$\sim 3.4 \times 10^{-4932}$	$\sim 1.1 \times 10^{4932}$

Os tamanhos podem variar conforme a plataforma. Os valores acima correspondem à arquitetura x86-64 com GCC ou Clang.

4.1.2 As Duas Regras Fundamentais

4.1.2.1 Regra 1: Em operações matemáticas, o maior vence

Quando dois tipos diferentes participam de uma operação aritmética, o compilador promove o menor para o maior antes de calcular. Nenhum dado é perdido nessa etapa.

```
%writefile cpp/src/04_exec_01.cpp
#include <iostream>
using namespace std;

int main() {

    double price {45.6};
    int units {10};

    // int é promovido para double antes da multiplicacao
    // o resultado é double, nenhum dado é perdido
    double total_price = price * units;

    cout << "Preco total: " << total_price << endl;
    // Saida: 456

    cout << "sizeof total_price: " << sizeof(total_price) << endl;
    // Saida: 8 (bytes de um double)

    return 0;
}
```

Overwriting cpp/src/04_exec_01.cpp

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_01.cpp -o cpp/bin/04_exec_01
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_01`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_01
```

Continuando, em **expressões encadeadas**, o compilador resolve **operação por operação**, promovendo conforme necessário. Analise o exemplo abaixo:

```
%writefile cpp/src/04_exec_02.cpp
#include <iostream>
using namespace std;

int main() {

    int    a {5};
    double b {2.7};
    int    c {3};
    double d {1.5};
    int    e {4};

    // Passo 1: a + b → int + double → double + double = 7.7
    // Passo 2: c * d → int * double → double * double = 4.5
    // Passo 3: 7.7 - 4.5 = 3.2
    // Passo 4: 3.2 + e → double + int → double + double = 7.2

    auto resultado = a + b - c * d + e;

    cout << "resultado: " << resultado << endl;
    // Saida esperada: 7.2

    cout << "sizeof resultado: " << resultado << endl;

    return 0;
}
```

Writing cpp/src/04_exec_02.cpp

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_02.cpp -o cpp/bin/04_exec_02
```

192.12s - pydevd: Sending message related to process being replaced timed-out after 5 seconds

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_02`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_02
```

4.1.2.2 Regra 2: Em atribuição, o destino manda

Quando o resultado de uma expressão é guardado em uma variável, o tipo da variável de destino prevalece. Se o tipo de destino for menor, o compilador trunca o valor, ou seja, a parte que não cabe é descartada sem arredondamento.

```
%writefile cpp/src/04_exec_03.cpp
#include <iostream>
using namespace std;

int main() {

    double y {45.99};
    int    x;

    // double é truncado para int
    // a parte decimal é descartada, nao arredondada
    x = y;

    cout << "O valor de x é: " << x << endl;
    // Saida: 45 (e nao 46)

    cout << "sizeof x: " << sizeof(x) << endl;
    // Saida: 4

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_03.cpp -o cpp/bin/04_exec_03
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_03`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_03
```

4.1.2.3 Modificador unsigned

Quando um tipo `signed` opera com um tipo `unsigned` de mesmo tamanho, o compilador converte o `signed` para `unsigned`. Isso significa que valores negativos são reinterpretados como números positivos enormes, e o resultado pode ser completamente inesperado.

```
%%writefile cpp/src/04_exec_04.cpp
#include <iostream>
using namespace std;

int main() {
    int a {-3};
    unsigned int b {5};

    // a é convertido para unsigned int
    // -3 em unsigned int vira 4294967293
    // a comparacao retorna false, e nao true como esperado
    if (a < b)
        cout << "a é menor" << endl;
    else
        cout << "resultado inesperado" << endl;
    // Saida: resultado inesperado

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_04.cpp -o cpp/bin/04_exec_04
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_04`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_04
```

4.2. Conversão Explícita (Casting)

Conversão explícita é quando o desenvolvedor decide, de forma intencional, transformar um valor de um tipo para outro. Diferente da conversão implícita, onde o compilador toma a decisão sozinho, aqui você assume o controle e a responsabilidade pelo resultado. A principal ferramenta para isso em C++ moderno é o `static_cast`.

A sintaxe do `static_cast` segue este formato.

```
static_cast<tipo_destino>(expressao)
```

O `static_cast` é verificado pelo compilador antes do programa rodar. Se os tipos não forem compatíveis, você recebe um erro de compilação claro, no momento certo. Isso garante que a conversão é intencional e segura dentro do que o padrão C++ permite.

4.2.1 Exemplo 1: Divisão Inteira

O caso mais comum onde `static_cast` é necessário é a divisão entre dois inteiros quando o resultado esperado é decimal.

```
%%writefile cpp/src/04_exec_05.cpp
#include <iostream>
using namespace std;

int main() {
    int x {10};
```

```
int y {4};

// Divisao inteira, resultado 2, a parte decimal é perdida
double errado = x / y;
cout << errado << endl; // Saida: 2

// static_cast converte x para double antes da divisao
// agora é double / int, o compilador promove y tambem
double correto = static_cast<double>(x) / y;
cout << correto << endl; // Saida: 2.5

return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_05.cpp -o cpp/bin/04_exec_05
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_05`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_05
```

A conversão acontece antes da operação. Ao converter `x` para `double`, o compilador aplica a regra da promoção aritmética e `y` também vira `double` automaticamente. A divisão então ocorre com precisão total.

4.2.2 Exemplo 2: Conversão entre int e char

Todo caractere em C++ tem um valor numérico correspondente definido pela tabela ASCII. O tipo `char` armazena internamente um número inteiro de 1 byte, e o que muda é apenas como esse valor é interpretado na saída. Isso torna a conversão entre `int` e `char` um exemplo direto e concreto de `static_cast`.

```
%writefile cpp/src/04_exec_06.cpp
#include <iostream>
using namespace std;

int main() {

    // int para char: o valor numerico é interpretado como caractere
    int codigo {65};
    char letra = static_cast<char>(codigo);
    cout << letra << endl; // Saida: A

    // char para int: revela o codigo ASCII do caractere
    char c {'a'};
    int ascii = static_cast<int>(c);
    cout << ascii << endl; // Saida: 97

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_06.cpp -o cpp/bin/04_exec_06
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_06`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_06
```

Sem o `static_cast`, imprimir um `int` com valor 65 sempre exibirá o número 65, nunca o caractere `A`. O cast instrui o compilador a tratar aquele valor como um `char` no momento da operação.

4.2.4 Exemplo 4: A ordem do cast muda o resultado

Este é um ponto que gera confusão. Aplicar `static_cast` na expressão inteira produz resultado diferente de aplicá-lo em cada operando separadamente.

```
%%writefile cpp/src/04_exec_07.cpp
#include <iostream>
using namespace std;

int main() {

    double x {12.5};
    double y {34.6};

    // Caso 1: cast em cada operando antes de somar
    // 12.5 vira 12, 34.6 vira 34, soma = 46
    int soma1 = static_cast<int>(x) + static_cast<int>(y);
    cout << soma1 << endl; // Saida: 46

    // Caso 2: soma primeiro, depois converte o resultado
    // 12.5 + 34.6 = 47.1, depois vira 47 (truncamento)
    int soma2 = static_cast<int>(x + y);
    cout << soma2 << endl; // Saida: 47

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_07.cpp -o cpp/bin/04_exec_07
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_07`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_07
```

Nos dois casos ocorre truncamento, sem arredondamento. A diferença está em quando o truncamento acontece. O exemplo abaixo reúne os três comportamentos em um único programa para facilitar a comparação.

```
%%writefile cpp/src/04_exec_08.cpp
#include <iostream>
using namespace std;

int main() {

    double x {12.5};
    double y {34.6};

    // Cast implícito: soma os doubles e converte o resultado para int
    // 12.5 + 34.6 = 47.1 -> truncamento -> 47
    int sum = x + y;
    cout << "A soma é: " << sum << endl; // Saida: 47
    cout << "O tamanho de soma: " << sizeof(sum) << endl; // Saida: 4

    // Cast explícito em cada operando antes de somar
    // 12.5 vira 12, 34.6 vira 34, depois 12 + 34 = 46
    sum = static_cast<int>(x) + static_cast<int>(y);
    cout << "A soma é: " << sum << endl; // Saida: 46
    cout << "O tamanho de soma: " << sizeof(sum) << endl; // Saida: 4

    // Cast explícito na expressão completa
    // 12.5 + 34.6 = 47.1 -> truncamento -> 47
    sum = static_cast<int>(x + y);
    cout << "A soma é: " << sum << endl; // Saida: 47
    cout << "O tamanho de soma: " << sizeof(sum) << endl; // Saida: 4

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_08.cpp -o cpp/bin/04_exec_08
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_08`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_08
```

O resultado que muda entre os três casos é sempre o `46` do segundo. Isso ocorre porque o truncamento acontece antes da soma, descartando a parte decimal de cada operando individualmente antes de qualquer cálculo.

4.2.4 Observação: Cast estilo C

O C++ herdou da linguagem C uma forma mais antiga de conversão explícita, que usa parênteses diretamente na frente do valor.

```
double PI {3.14};
int int_pi = (int) PI; // Estilo C
```

Essa sintaxe não é proibida e você vai encontrá-la em código legado. O problema é que o compilador não verifica se a conversão faz sentido antes do programa rodar, ele simplesmente executa sem reclamar. Em C++ moderno, a recomendação é sempre usar `static_cast`. A tabela abaixo resume as diferenças.

Característica	Cast estilo C <code>(int)x</code>	<code>static_cast<int>(x)</code>
Verificado pelo compilador	Não	Sim
Fácil de localizar no código	Não	Sim
Intenção clara	Não	Sim
Pode executar cast perigoso silenciosamente	Sim	Não

4.3 Overflow e Underflow

Cada tipo de dado possui uma capacidade fixa de armazenamento determinada pelo número de bytes que ocupa. Quando operamos com valores que ultrapassam esses limites, o programa não gera um erro, ele simplesmente descarta o que não cabe e continua executando com um valor incorreto. Esse comportamento silencioso é o que torna **overflow** e **underflow** especialmente perigosos.

- **Overflow** ocorre quando tentamos armazenar um valor maior que o máximo suportado pelo tipo. O valor não cabe na memória disponível e o dado estoura para o outro extremo do intervalo.
- **Underflow** ocorre quando tentamos armazenar um valor menor que o mínimo suportado pelo tipo. O valor cai abaixo do limite inferior e volta para o extremo superior do intervalo.

Esses fenômenos podem ocorrer com qualquer tipo de dado, sejam inteiros, ponto flutuante, com ou sem sinal. O que determina quando o problema aparece é o intervalo suportado pelo tipo escolhido. A tabela abaixo mostra os limites dos tipos primitivos sem modificadores.

Tipo Completo	Tamanho	Valor Mínimo	Valor Máximo
<code>bool</code>	1 byte	0 (false)	1 (true)
<code>char</code>	1 byte	-128	127
<code>signed char</code>	1 byte	-128	127
<code>unsigned char</code>	1 byte	0	255
<code>short</code>	2 bytes	-32.768	32.767
<code>unsigned short</code>	2 bytes	0	65.535
<code>int</code>	4 bytes	-2.147.483.648	2.147.483.647
<code>unsigned int</code>	4 bytes	0	4.294.967.295
<code>long</code>	4 ou 8 bytes	-2.147.483.648	2.147.483.647
<code>unsigned long</code>	4 ou 8 bytes	0	4.294.967.295
<code>long long</code>	8 bytes	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
<code>unsigned long long</code>	8 bytes	0	18.446.744.073.709.551.615
<code>float</code>	4 bytes	$\sim 1,2 \times 10^{-38}$	$\sim 3,4 \times 10^{38}$
<code>double</code>	8 bytes	$\sim 2,3 \times 10^{-308}$	$\sim 1,7 \times 10^{308}$
<code>long double</code>	10 a 16 bytes	$\sim 3,4 \times 10^{-4932}$	$\sim 1,1 \times 10^{4932}$

Nos exemplos a seguir usamos `unsigned char` por conveniência didática. Com apenas 1 byte e intervalo de 0 a 255, seus limites são atingidos com operações simples, o que torna o comportamento de overflow e underflow fácil de observar e acompanhar.

4.3.1 A base do problema: capacidade de bits

Todo tipo de dado tem um limite de representação definido pelo número de bits que ocupa. Por exemplo, um `unsigned char` ocupa 1 byte, ou seja, 8 bits, o que permite representar 2^8 combinações distintas, resultando em um intervalo de 0 a 255. Por ser o menor tipo inteiro disponível, ele é ideal para ilustrar o problema, pois seus limites são atingidos com operações simples e fáceis de acompanhar.

Tipo	Tamanho	Mínimo	Máximo	Total de valores
unsigned char	1 byte	0	255	256

Quando somamos dois valores cujo resultado ultrapassa o máximo do tipo, o que acontece em nível de bits é que todos os bits são zerados e a contagem recomeça do zero. O valor excedente é calculado pelo resto da divisão por 256 (para `unsigned char`).

4.3.2 Exemplo 1: Overflow

```
%writefile cpp/src/04_exec_09.cpp
#include <iostream>
using namespace std;

int main() {

    unsigned char char_var {56};
    unsigned char val1 {130};
    unsigned char val2 {131};

    // 130 + 131 = 261
    // 261 nao cabe em unsigned char (max 255)
    // 261 - 256 = 5, o que sobra da operacao binaria
    char_var = val1 + val2;

    cout << "char_var: " << static_cast<int>(char_var) << endl; // Saida: 5

    return 0;
}
```

Writing cpp/src/04_exec_09.cpp

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_09.cpp -o cpp/bin/04_exec_09
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_09`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_09
```

O que ocorre em nível de bits é exatamente isso: ao chegar em `11111111` (255 decimal), o próximo incremento zera todos os bits e reinicia em `00000000` (0 decimal), depois `00000001` (1 decimal), e assim por diante.

```
11111101 (253 dec)
11111110 (254 dec)
11111111 (255 dec)
    ↓ Reseta todos os bits
00000000 (000 dec)
00000001 (001 dec)
```

4.2.3 Exemplo 2: Underflow

O underflow é o movimento inverso. Quando subtraímos além do limite inferior, o valor volta para o extremo superior do intervalo.

```
%writefile cpp/src/04_exec_10.cpp
#include <iostream>
using namespace std;

int main() {

    unsigned char char_var {55};

    unsigned char val1 {130};
    unsigned char val2 {131};

    // 130 - 131 = -1
    // -1 nao existe em unsigned char
    // 256 - 1 = 255, o valor vai para o extremo superior
    char_var = val1 - val2;

    cout << "char_var: " << static_cast<int>(char_var) << endl; // Saida: 255
```

```
    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_10.cpp -o cpp/bin/04_exec_10
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_10`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_10
```

4.3.4 Exemplo completo

O exemplo abaixo parte de 250 e incrementa até cruzar o limite de 255, depois parte de 1 e decrementa até cruzar o limite inferior. Isso permite observar exatamente em qual passo o overflow e o underflow acontecem.

```
%%writefile cpp/src/04_exec_11.cpp
#include <iostream>

int main() {

    //Overflow
    unsigned char data {250};

    ++data;
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 251
    ++data;
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 252
    ++data;
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 253
    ++data;
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 254
    ++data;
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 255
    ++data; // Overflow: 255 + 1 volta para 0
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 0

    // Underflow
    data = 1;
    --data;
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 0
    --data; // Underflow: 0 - 1 vai para 255
    std::cout << "data : " << static_cast<int>(data) << std::endl; // 255

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/04_exec_11.cpp -o cpp/bin/04_exec_11
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/04_exec_11`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/04_exec_11
```

4.3.5 Resumo

Os dois casos abaixo sintetizam o comportamento de overflow e underflow em uma única variável. Atribuir 261 a um `unsigned char` resulta em 5, pois 261 excede o limite de 255 e o excedente reinicia a contagem do zero. Atribuir -1 resulta em 255, pois o valor cai abaixo do mínimo e o sistema volta pelo extremo superior.

```
unsigned char char_var {55};
```

```
char_var = 261; // Overflow: 261 - 256 = 5  
char_var = -1;  // Underflow: 256 - 1 = 255
```

Em ambos os casos o compilador não emite erro. O programa compila e executa com valores silenciosamente incorretos.